

Disponibilidade e Escalabilidade de Serviços

Infraestruturas de Computação na Nuvem, Aula 08

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- Disponibilidade e escalabilidade como atributos de qualidade.
- Escalabilidade vertical vs horizontal.
- Serviços com e sem estado (*stateless* vs *stateful*).
- Auto-scaling: métricas e políticas.
- Balanceamento de carga: algoritmos.

Agenda (2/2)

- Load balancing de camada 4 vs camada 7, e como se combinam à escala.
- Balanceamento global: DNS e edge locations.
- Health checks e remoção automática de instâncias.
- Estratégias de implantação sem downtime.
- Caching e CDNs.
- Limites da escalabilidade: a lei de Amdahl.
- Escalabilidade e auto-scaling no Kubernetes.

Disponibilidade e Escalabilidade como Atributos de Qualidade

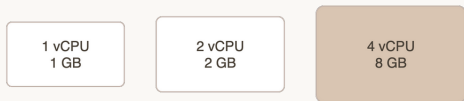
- Retomando a Aula 02: SLAs definem compromissos de disponibilidade (ex. 99,9%) associados a Tiers de datacenter.
- Disponibilidade: capacidade de o serviço responder corretamente, mesmo com falhas parciais.
- Escalabilidade: capacidade de o serviço acompanhar o crescimento da procura sem degradar o desempenho.
- Ambos os atributos dependem de arquitetura, não apenas de hardware mais potente.

Escalabilidade Vertical vs Horizontal

- **Scale-up (vertical)**: aumentar os recursos de uma única instância (mais CPU, mais RAM).
- Limite físico e custo crescente; ponto único de falha mantém-se.
- **Scale-out (horizontal)**: acrescentar mais instâncias a correr em paralelo.
- Requer aplicações preparadas para correr em múltiplas instâncias (sem estado local crítico).

Crescer para Cima ou para o Lado

Escalabilidade vertical (scale-up)



Máquina maior. Simples, mas tem limite físico e mantém o ponto único de falha.

Escalabilidade horizontal (scale-out)



Mais instâncias em paralelo. Exige aplicação sem estado local crítico.

Serviços Sem Estado e Com Estado

- **Stateless**: cada pedido é independente; qualquer instância pode responder a qualquer pedido.
- É esta propriedade que torna a escalabilidade horizontal simples – acrescentar instâncias não exige coordenação.
- **Stateful**: a instância guarda estado relativo ao cliente (ex. sessão em memória), pelo que os pedidos seguintes têm de voltar à mesma.
- Estratégia habitual: externalizar o estado para um armazenamento partilhado (base de dados, cache distribuída como Redis).
- **Session affinity** (ou *sticky sessions*): alternativa em que o balanceador envia sempre o mesmo cliente para a mesma instância – mais simples, mas prejudica a distribuição de carga e a tolerância a falhas.

Auto-Scaling: Métricas

- Decisão de escalar (para cima ou para baixo) baseada em métricas monitorizadas continuamente.
- Utilização de **CPU** e de memória das instâncias existentes.
- **Latência** das respostas ao utilizador final.
- Tamanho de **filas** de pedidos ou de mensagens pendentes.

Auto-Scaling: Políticas Reativas vs Preditivas

- **Reativas:** escalar quando uma métrica ultrapassa um limiar (ex. CPU > 70%).
- Simples de implementar, mas reagem depois do problema já estar a acontecer.
- **Preditivas:** antecipam a procura com base em padrões históricos (ex. hora de ponta diária).
- Reduzem o atraso de arranque de novas instâncias, mas exigem modelos e histórico de dados.

Balanceamento de Carga: Algoritmos (1/2)

- Um **load balancer** distribui pedidos por um conjunto de instâncias equivalentes.
- **Round-robin**: distribui pedidos sequencialmente, um a um, por cada instância.
- **Least connections**: envia o próximo pedido para a instância com menos ligações ativas.

Balanceamento de Carga: Algoritmos (2/2)

- **Hashing consistente:** usa uma função de hash (ex. sobre o IP do cliente) para mapear pedidos sempre à mesma instância.
- Útil para manter afinidade de sessão sem necessitar de partilha de estado entre instâncias.
- A escolha do algoritmo depende do padrão de carga e da necessidade (ou não) de afinidade.

Load Balancing: Camada 4 vs Camada 7

- **Camada 4** (transporte, TCP/UDP): decide o encaminhamento com base em IP e porta, sem olhar ao conteúdo.
- Mais rápido e simples, mas com pouca informação sobre a aplicação.
- **Camada 7** (aplicação, HTTP): decide com base em conteúdo – URL, cabeçalhos, cookies.
- Permite routing mais inteligente (ex. por caminho do pedido), ao custo de mais processamento.

Balanceamento em Camadas: um Caso Real

- À escala, camada 4 e camada 7 não são alternativas – combinam-se em cascata. Exemplo documentado publicamente pela Facebook (Engineering, 2015):
- Um único servidor aplicacional satura depressa → acrescenta-se um balanceador **L7** (decide por conteúdo, ex. proxygen) à frente de várias instâncias.
- O L7LB também satura → acrescenta-se um balanceador **L4** (ex. IPVS) à frente de vários L7LBs, distribuído por **ECMP** (Equal-Cost Multi-Path) ao nível do router.
- O mesmo padrão repete-se: dezenas de servidores por cluster, dezenas de clusters por datacenter, múltiplos datacenters – cada nível resolve um gargalo do nível anterior, não o problema todo de uma vez.

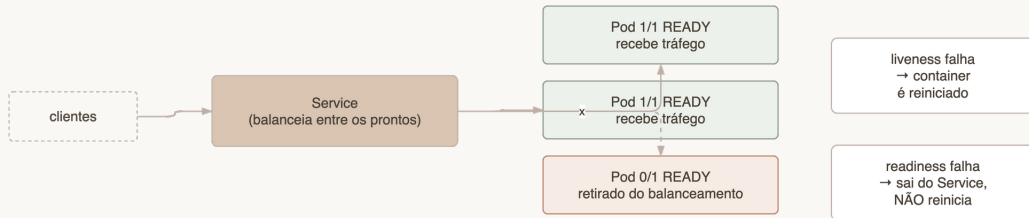
Balanceamento Global: DNS e Edge Locations

- Antes de qualquer balanceador L4/L7 ser contactado, o cliente resolve um nome DNS – esse é o primeiro (e mais grosseiro) nível de balanceamento.
- **DNS geográfico/por latência:** o servidor DNS responde com o IP do *edge* ou datacenter mais próximo do cliente, não sempre o mesmo IP para todos.
- “Mais próximo” não é só distância geográfica: sistemas como o Cartographer (Facebook) medem latência real entre resolvers de ISPs e cada *edge*, porque a rota mais curta no mapa nem sempre é a mais rápida.
- Impacto concreto: um pedido HTTPS Seul-Oregon direto pode demorar ~ 600 ms só no *handshake*; passando por um edge em Tóquio mais próximo, cai para ~ 240 ms – o mesmo raciocínio das Regiões e Zonas de Disponibilidade da Aula 02.
- Este nível global é o que torna as CDNs (adiante) eficazes: aproxima o ponto de

Health Checks

- O load balancer verifica periodicamente se cada instância está saudável (ex. pedido HTTP a `/health`).
- Instâncias que falham os health checks são removidas automaticamente do conjunto de balanceamento.
- Quando voltam a responder corretamente, são reintegradas sem intervenção manual.
- Mecanismo essencial para disponibilidade: falhas parciais não afetam o utilizador final.

Readiness vs Liveness

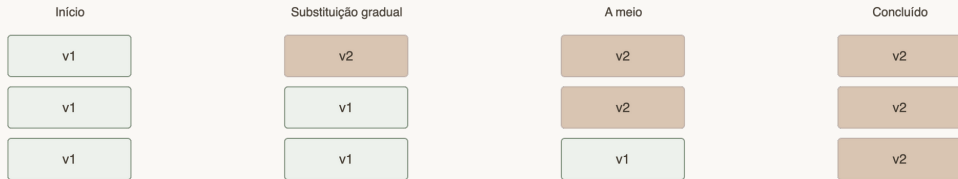


O serviço continua disponível enquanto houver réplicas prontas.

Estratégias de Implantação Sem Downtime

- **Rolling update:** substituem-se as instâncias gradualmente, algumas de cada vez, mantendo o serviço disponível.
- **Blue-green:** mantêm-se dois ambientes completos; o tráfego é comutado de uma vez do antigo para o novo, permitindo reverter depressa.
- **Canary:** encaminha-se uma pequena fração do tráfego para a nova versão, aumentando-a progressivamente se não houver erros.
- Todas dependem de health checks fiáveis para detetar problemas antes de afetarem todos os utilizadores.
- O Kubernetes implementa o rolling update por omissão nos Deployments (Aula 05).

Rolling Update, Passo a Passo



O Pod novo só recebe tráfego depois de passar a readiness probe;
só então o antigo é removido. Por isso o cliente nunca vê erro.

Caching e Redes de Distribuição de Conteúdo

- Escalar nem sempre significa acrescentar instâncias: evitar trabalho repetido é frequentemente mais eficaz.
- **Cache** em memória (ex. Redis, Memcached): guarda resultados de operações dispendiosas, reduzindo a carga sobre a base de dados.
- **CDN** (Content Delivery Network): replica conteúdo estático por servidores geograficamente distribuídos, próximos dos utilizadores.
- Benefícios: menor latência para o utilizador e menor carga sobre a infraestrutura de origem.
- Custo: gerir a *invalidação* da cache – garantir que conteúdo desatualizado não é servido indefinidamente.

Limites da Escalabilidade: a Lei de Amdahl

- Nem todos os sistemas escalam linearmente com o número de instâncias acrescentadas.
- A **lei de Amdahl** estabelece que o ganho máximo é limitado pela fração do trabalho que *não* pode ser paralelizada.
- Se 10% do trabalho for inerentemente sequencial, o ganho máximo teórico fica limitado a cerca de 10 vezes, por muitas instâncias que se acrescentem.
- Em sistemas distribuídos, o gargalo é muitas vezes um recurso partilhado – tipicamente a base de dados.
- Implicação prática: acrescentar instâncias sem eliminar o estrangulamento comum traz retornos rapidamente decrescentes.

Escalabilidade no Kubernetes

- **Escalamento manual:** alterar o número de réplicas de um Deployment (`kubectl scale`), como fizemos na Aula 05.
- **HPA** (Horizontal Pod Autoscaler): ajusta automaticamente o número de Pods com base em métricas como a utilização de CPU.
- O HPA depende do **metrics-server** estar instalado no cluster para obter essas métricas.
- **VPA** (Vertical Pod Autoscaler): ajusta os *requests* e *limits* de cada Pod, em vez do número de réplicas.
- **Cluster Autoscaler:** acrescenta ou remove *nós* ao cluster quando os existentes ficam sem capacidade.

Síntese da Aula (1/2)

- Disponibilidade e escalabilidade são atributos de qualidade ligados a SLAs e Tiers.
- Escalar verticalmente tem limites; escalar horizontalmente exige arquitetura adequada.
- Auto-scaling usa métricas (CPU, latência, filas) e políticas reativas ou preditivas.

Síntese da Aula (2/2)

- Balanceamento de carga distribui pedidos: round-robin, least connections, hashing consistente. Camada 4 é mais rápida; camada 7 decide pelo conteúdo – e, à escala, combinam-se em cascata (L4 à frente de L7).
- O primeiro nível de balanceamento é o DNS: resolve o cliente para o datacenter/edge mais próximo em latência real, não apenas em distância no mapa.
- Health checks garantem que só instâncias saudáveis recebem tráfego, e sustentam as estratégias de implantação sem downtime (rolling, blue-green, canary).
- Caching e CDNs escalam evitando trabalho repetido; a lei de Amdahl lembra que há sempre um limite imposto pela parte não paralelizável.
- No Kubernetes, o HPA automatiza o escalamento horizontal e o Service faz o balanceamento interno.

Obrigada.

Questões e comentários